

Arquitetura Inicial — Gestão de Materiais

Gestão de Materiais Corporativo

Gerado em 26/09/2026

Fonte: arquitetura-inicial.md

1. Decisão de arquitetura

O sistema será um monorepo com frontend e backend desacoplados, seguindo o padrão de monólito modular:

frontend/	→ Next.js, React e TypeScript
backend/	→ Django + Django REST Framework
ai-service/	→ FastAPI para IA futura
infra/	→ configurações de deploy e serviços externos
docs/	→ documentação do produto e arquitetura

O Django será a fonte de verdade para autenticação, autorização, regras de negócio e persistência. O Next.js será responsável pela experiência web. O serviço de IA será separado para evitar que processamento pesado comprometa os fluxos transacionais.

2. Módulos planejados

1. Identidade e acesso
2. Organização, filiais, áreas e departamentos
3. Catálogo de materiais
4. Locais e almoxarifados
5. Estoque e movimentações
6. Solicitações internas
7. Aprovações
8. Compras e cotações
9. Fornecedores
10. Recebimento e notas fiscais
11. Entregas e distribuição
12. Ativos e custódia
13. Manutenção
14. Inventário
15. Relatórios
16. Notificações
17. Auditoria e configurações

3. Identidade e escopo de acesso

Cada pessoa terá seu próprio usuário. Não haverá contas compartilhadas por área.

A autorização terá duas camadas:

Permissão funcional

Define o que o usuário pode fazer:

- materials.view
- materials.create
- materials.update

- stock.receive
- stock.issue
- stock.transfer
- requests.create
- requests.approve
- purchases.manage
- reports.export
- users.manage

Escopo de dados

Define onde o usuário pode atuar:

- própria área;
- próprio departamento;
- própria filial;
- determinados locais de estoque;
- todas as áreas.

A API deve aplicar o escopo nas consultas e nas mutações. Esconder um botão no frontend nunca será considerado controle de segurança.

4. Perfis iniciais

- Administrador da plataforma
- Administrador da organização
- Gestor de área
- Analista de área
- Estoquista
- Comprador
- Solicitante
- Auditor

Os perfis serão combináveis por usuário quando necessário, mas o acesso final será sempre a interseção entre permissões e escopo organizacional.

5. Áreas iniciais

- TI
- Suprimentos
- Administrativo
- RH

As áreas serão dados configuráveis, não condicionais fixos espalhados pelo código.

6. Regras não negociáveis

1. O estoque não pode ficar negativo.
2. Movimentações de estoque são somente inclusão; correções geram estorno ou ajuste.
3. Uma aprovação deve guardar usuário, data, decisão e justificativa.
4. Uma solicitação aprovada não pode ser alterada sem nova avaliação.
5. Itens individualizados não podem ter duas custódias ativas.
6. Toda ação sensível deve gerar auditoria.
7. Route Handlers e Server Actions validam todas as permissões no servidor.
8. Dados excluídos logicamente não podem desaparecer dos relatórios de auditoria.
9. Uploads devem ser validados por tipo, tamanho e armazenamento seguro.
10. Processos idempotentes não podem criar entradas ou saídas duplicadas.

7. Primeira versão do domínio

Entidades da fundação:

```
organizations
branches
departments
areas
users
roles
permissions
role_permissions
user_roles
user_scopes
audit_logs
```

Entidades da próxima etapa:

```
materials
material_categories
warehouses
stock_balances
stock_movements
stock_reservations
```

8. Padrão de API

A API usará respostas JSON consistentes:

```
{
  "data": {},
  "meta": {},
  "errors": []
}
```

Erros de validação retornarão campos específicos. Erros de autorização retornarão 403. Recursos inexistentes retornarão 404. Operações duplicadas deverão ser protegidas por chave de idempotência quando alterarem estoque, compras ou entregas.

9. Estratégia de testes

Cada módulo deverá ter:

- testes de regra de negócio;
- testes de autorização;
- testes de integração com banco;
- testes de API;
- testes de interface para fluxos críticos.

O primeiro gate será o fluxo: criar usuário → atribuir área → fazer login → acessar apenas os recursos permitidos.